

Symbolic Computation and Teaching

Dana S. Scott

*School of Computer Science
Carnegie Mellon University
Pittsburgh, Pennsylvania 15213, USA*

e-mail: dana.scott@cs.cmu.edu

Abstract. Since 1989, the author has tried to show that it is possible to put a complete semester-long course in machine-held form. Several examples have been carried out (in *Mathematica*), and the paper reports on the experience, on the problems encountered, and on some suggestions for future developments.

■ 1. Twenty Questions

In his keynote address at the **Second Annual Conference on Technology in Collegiate Mathematics** in November 1989 at The Ohio State University (see the newsletter UME TRENDS, for January 1990), Professor Lynn Steen effectively spoke for parents and students, scientists and engineers, colleagues and administrators by raising the following *twenty questions for calculus reformers*. Steen suggested that responding to these questions could form an agenda for the current work of people exploring the use of computers in curricular reform. Though several years have passed since Steen wrote these words, the questions remain highly relevant, since satisfactory conclusions about the use of computers have still not been reached.

In looking at the questions, remember they are addressed to mathematics departments, not to computer science departments. Calculus reform is still today a most controversial topic because so many students are required to take calculus, but many mathematicians (in the United States) feel that “reform” has meant “dumbing-down” to a level where the preparation of students in mathematics — both at school and college — is being positively harmed. This is a very broad topic, however, and we are only going to address here some of the problems about using computers and symbolic computation in various other courses with mathematical content, and we cannot enter here into the continuing battle over the future of the Calculus. All the issues are connected, however.

Steen’s questions can be broken into five sections:

□ Learning

1. Can computers help students understand mathematics?
2. Can students develop mathematical intuition without performing extensive mathematical manipulations?
3. Do the mechanics of computing obscure mathematical insight?
4. Will using computers reduce students’ facility to compute by hand?

□ Curriculum

5. How does computing change what students should know about mathematics?
6. How does computing change what students can learn about mathematics?
7. Where in the curriculum is computing most appropriate?
8. Will use of computers reduce the need for remediation?

□ Resources

9. Can colleges afford computers for all mathematics students?
10. How much time and distraction is computing worth?
11. When will there be good software and compatible hardware?
12. Can textbooks ever reflect contemporary computer examples?

□ Teaching

13. How much programming should be taught in mathematics courses?
14. Can pure mathematicians convey an appropriate computational perspective?
15. How will new faculty fit into computer-enhanced programs?
16. Will use of computers improve teaching of mathematics?

□ Dilemmas

17. Won't computer packages for calculus lead, as they have in statistics, to much meaningless calculation?
18. If computers handle routine calculations, what will students do instead?
19. What are appropriate prerequisites for computer-based calculus courses?
20. Should mathematics be a lab science?

The author has many answers to and opinions on these questions. Providing some answers and comments will form the main theme of this paper. The bottom line is that, yes, I personally believe that mathematics *should be in part* a laboratory science — the problem in the past has been that we have not had sufficiently powerful tools available to do the necessary experimentation on a large scale. Of course, the computing machine does not replace *imagination* — nor does the laboratory in any other science. Neither does the machine replace the standard means of *exposition* — but it can make the composition and presentation of books and lectures easier and more vivid, and more flexible. As with any tool, considerable effort is required in learning to use it effectively. The financial investment for the institution is considerable as well. The major question to be discussed here is whether the money and effort is worth the gain.

■ 2. The Author's Experience

What follows is a brief chronological survey of the courses in which the author and some of his associates have been involved. Then the syllabi of the *projective geometry* and *discrete mathematics* course will be given later in this section in some detail — especially to emphasize the point that it is possible to produce a substantial semester-long course completely in a computer-based format.

Projective Geometry, Fall, '89, CMU [Scott]
Automata Theory, '90, '91, '93, Stevens Institute for Technology [Sutner]
Topics in Discrete Mathematics, Spr. '91, CMU [Scott]
Projective Geometry, Spr., '93, RISC-Linz [Scott]
Automata Theory, Spring '93, RISC-Linz [Sutner]
Introduction to ModMath, Spr. '94, CMU [Scott, Miller]
Introduction to ModMath, Fall '94, CMU [Scott, Miller]
Problem Solving, Spr. '95 CMU [Scott, Miller, Sleator, Tygar]
Introduction to ModMath, Spr. '95, CMU [Miller, Statman]
Introduction to ModMath, Fall '95, CMU [Albert, Miller, Sutner]
Introduction to ModMath, Spr. '96, CMU [Scott, Miller, Sutner,
 Walkington]

As indicated, the first course attempted was a course in projective geometry during the fall semester of 1989 at Carnegie Mellon (CMU). This was followed by a "topics" course, where the students carried out various projects in discrete mathematics and in using computer graphics. During a sabbatical year in Austria at RISC-Linz ('92/'93), the author gave another, improved version of the projective geometry course. That same semester Klaus Sutner also gave at Linz a new version of his course on finite automata, which he had developed over several years at the Stevens Institute for Technology. These courses were presented in lectures with the instructor using a projector from a computer, and sessions were held for students in a computer cluster. We were very indebted to Professor Bruno Buchberger for the opportunity to set up the computer classroom and to deliver these two courses. The very helpful staff and students at RISC-Linz were also essential for making things work.

After returning to the States, Scott began development of an introductory course in discrete mathematics for first-year students with the assistance of Philip L. Miller, head of the Introductory Programming Group at CMU. The computer-based classrooms used for programming were employed for the laboratory sessions, and lectures were given in classrooms with a projector. A more advanced course in discrete mathematics was also tried out with the cooperation of other CS faculty. Sutner then joined the teaching faculty in the Introductory Programming Group in Computer Science at CMU in the fall of 1995. The introductory course had also been rerun twice and a complete revision was made for the Spring Semester 1996, with the cooperation of Miller and Sutner and other CMU faculty.

Some acknowledgments are in order. Over the years, Scott has been very ably assisted at various times by four of his former students: Jean-Philippe Vidal (Paris), Marko Petkovsek (Ljubljana, Slovenia), and J. Todd Wilson (Cornell), Drew Dean (Princeton), and by one post-doctoral visitor, Dr. Christine Luksch (Darmstadt). He is much indebted to them. Essential advice about *Mathematica* and about computer-based teaching has been obtained at various times from Prof. John Gray (Illinois) and from many people at Wolfram Research, Inc. (Urbana, Illinois), including Stephen Wolfram, Theo Gray, Roman Maeder, Igor Rivin, Henry Cejtin, Cameron Smith, and Nancy Blachman. However, without the extensive involvement, both in planning and in execution, by Philip Miller and Klaus Sutner, the later courses could never have been conceived, mounted or completed. The two of them have been wonderful collaborators and friends, and we hope to continue joint work with new teaching developments in the future.

□ Syllabus for Projective Geometry with *Mathematica*

The course was divided into six chapters, with the objective to end up with the projective classification (over the complex numbers) of *cubic curves*. Starting from scratch, this is a heavy job for one semester!

Chapter 1: A review of some basic algebra. Complex numbers and Argand diagrams; roots of an n -th order polynomial equation and the roots of unity; determinants, the line joining two points, and areas of polygons; points in space, the intersection of two lines, and projections between intersecting planes; vector spaces and polynomial algebra.

Chapter 2: Homogeneous coordinates. Homogeneous coordinates for points and lines; projective representations of points and lines; dual spaces and bilinear pairing; linear forms and the dual pairing; exterior products of forms; meets and joins; computations and geometric interpretation; points on a line and concurrent lines; Desargues' Theorem and computing the intersection of lines determined by points; Theorems of Menelaus, Ceva and Pappus; the uniqueness of the 4th harmonic and grids of squares.

Chapter 3: Projective transformations. Matrices for transformations and point transformations; transformation by substitution and composition of transformations; line transformations; transforming quadrangles and quadrilaterals, and mapping triangles, quadrangles and quadrilaterals; the notion of crossratio and positioning points on a line; preservation of crossratio by transformation; perspective correspondences and compositions of perspectives; classifying transformations by fixed points and fixed lines.

Chapter 4: Polynomial duality. Curves, envelopes, and intersections; conic sections and quadratic equations; intersecting lines with curves and curves with curves; polydifferentiation and the ring of differential operators; evaluation through polydifferentiation; duality at higher degree and the dual vector spaces of polynomials; linear combinations of powers of linear forms; contact, multiplicity, and tangency; polydifferential graded algebra for "general" polynomials; degenerate curves; the Hessian, and testing conics for degeneracy; the tangents at a point on a curve; intersecting lines with a curve from an $(n-1)$ -tuple point.

Chapter 5: Conics. Poles and polars with respect to a conic; degenerate and non-degenerate conics and the relation to the Hessian; preservation of crossratios and the crossratio of four points on a conic; meets of conjugate lines, the envelope of a conic; the conic determined by five points; Pascal's Theorem; projectivities on a conic; conics determined by points and tangents; conics through four points; harmonic properties of conjugate points; involutions on a conic; the eight tangents at the intersection of two conics; pencils of conics.

Chapter 6: Cubics and some higher-degree curves. Special points on cubics; independence of points; nodes and cusps; the invariance of the Hessian determinant; preserving polydifferentiation by projective transformations; inflection points of cubics; cubics intersecting at eight points; normal form for cuspidal cubics, for nodal cubics, and classifying non-singular cubics; the configuration of inflection points of a non-singular cubic; graphing and parameterizing curves in general, and the parameterization of the non-singular cubic; the maximum number of double points on a curve; cataloging degenerate cubics; parametrization of a quartic curve; a few examples of other curves; a rational curve with positive deficiency; finding the tangential equation of a curve; analyzing rational functions.

□ Commentary

As this was the first course attempted, and as the number of students that have actually taken it so far (about 12) has been very small, it is as of yet not fully developed as the equivalent of a standard text book. The topics and methods of symbolic computation have been well tested, but far more commentary needs to be written and a better development of graphical illustrations to accompany the examples is needed. More exercises also have to be developed. The files that exist now, unfortunately, could not be used for self-study. If there is ever another opportunity to present this course, the author hopes to put it into publishable form, as he feels the mathematical approach adopted making use of computer algebra in several different ways has the potential for further development. We do not have the space to argue this point in any detail here, however.

The topic of the course is that of algebraic curves in the classical complex projective plane. Algebraic geometry in projective form standardly employs homogeneous polynomials. A polynomial in variables $\mathbf{xyz}$ is said to be *homogeneous* if, when multiplied out as a sum of monomials, all the monomials have the *same degree*. In the plane, the zeros of a homogeneous $\mathbf{xyz}$ -polynomial represent an algebraic curve. The dual, $\mathbf{uvw}$ -polynomials, in a dual set of variables, represent the tangential form of curves, that is to say envelopes.

How can the full impact of duality — familiar from elementary projective geometry as the duality between points and lines — be extended to curves of higher degree? In order to derive the necessary formulae it is most convenient to introduce the ring of *differential operators*. If we consider the space of all $\mathbf{xyz}$ -polynomials as a space of (continuous) infinitely differentiable multivariate functions, then we know that on such functions, $\mathbf{f}$, the operators of partial differentiation, $\partial/\partial \mathbf{x}$, $\partial/\partial \mathbf{y}$, and $\partial/\partial \mathbf{z}$, which are written as $\mathbf{D}[\mathbf{f}, \mathbf{x}]$, $\mathbf{D}[\mathbf{f}, \mathbf{y}]$, and $\mathbf{D}[\mathbf{f}, \mathbf{z}]$ in *Mathematica* notation, are associative and commutative (under composition of operators). As is well known, these compositions are also linear operators, and any linear combination of linear operators is again a linear operator. Moreover, composition of linear operators distributes over all linear combinations of operators; hence, the differential operators generate a *commutative ring of operators*.

In the light of these considerations, it is possible to define a ring homomorphism from the (free) ring of $\mathbf{uvw}$ -polynomials to these operators. (This homomorphism is actually an isomorphism.) In particular, if $\mathbf{f}$ is an $\mathbf{xyz}$ -polynomial and $\mathbf{g}$ is a $\mathbf{uvw}$ -polynomial of *lower* degree, then we can notate by $\mathbf{PD}[\mathbf{f}, \mathbf{g}]$ the result of letting $\mathbf{g}$ operate on $\mathbf{f}$ as a differential operator by calling up this homomorphism. However, differentiation between $\mathbf{xyz}$ -polynomials and $\mathbf{uvw}$ -polynomials can be made *symmetric* or *dual* in specific sense. Indeed, if $\mathbf{f}$ is a homogeneous $\mathbf{xyz}$ -polynomial, and $\mathbf{g}$ is a homogeneous $\mathbf{uvw}$ -polynomial, then $\mathbf{PD}[\mathbf{f}, \mathbf{g}]$, an operation the author calls *polydifferentiation*, may be defined as the result of allowing the polynomial of lower degree to operate on the polynomial of higher degree as a differential operator. In other words, the symbol $\mathbf{u}$ can be regarded as the operator $\partial/\partial \mathbf{x}$, or the symbol $\mathbf{x}$ can be regarded as the operator $\partial/\partial \mathbf{u}$, and similarly for other variables. In the case of polynomials of *equal degree*, by reference to monomials, it is easy to see that in evaluating $\mathbf{PD}[\mathbf{f}, \mathbf{g}]$ it makes no difference which is taken to be the operator and which is the operand.

The *linear homogeneous xyz*-polynomials, f , can be regarded as *lines*; while the linear uvw -polynomials, g , can be taken as points (up to a constant, non-zero factor, as with homogeneous coordinates). Then the equation $PD[f, g] == 0$ means that the point g *lies on* the line f . (In the linear case, partial differentiation works just like a dot product between vectors.) The equation also means the line f *passes through* the point g , and this remark is the basis for the explanation of the duality between points and lines. What about higher degrees? What we need is a special case of a theorem of Euler which tell us that, if f is a homogeneous xyz -polynomial of degree n , and if g is taken as the *linear form* $a u + b v + c w$, then we have the equation $PD[f, g^n] == n! f[a, b, c]$, where $f[a, b, c]$ is the evaluation of the polynomial as a function of xyz at abc . Among other consequences, this theorem tells us that the equation $PD[f, g^n] == 0$ means that the point g *lies on the curve* f . But there are many algebraic consequences. For example, by a simple use of the Binomial Theorem, we can argue that if g represents an m -tuple point of the curve f , then the polynomial $P[f, g^{n-m}]$ of degree m factors into the m linear factors representing the *tangents* to the curve f at the point g . The trick here is that pairing $PD[f, g]$ is a bilinear pairing between the vector spaces of n -degree homogeneous xyz -polynomials and n -degree homogeneous uvw -polynomials that makes then *dual vector spaces*. But the polynomials are not just elements of vector spaces, since they are parts of a pair of commutative rings, and PD has many simple properties with respect to the ring structure. A suitable axiomatization of these properties makes the ring of xyz -polynomials fully dual to the ring of uvw -polynomials, and the formal operations — as indicated — can have geometric interpretations.

Mathematica well implements commutative algebra (and many algebraic algorithms) and of course it implements partial differentiation. It does not implement a algebra of differential operators, however. The author was able to do that by defining the function $DegP$ for computing the degree of homogeneous polynomials, and by transcribing the axiomatic characterization of PD directly into rewrite rules between the two kinds of polynomials using *Mathematica* rule sets. These rules incorporate in symbolic form the rules of partial differentiation, and so *Mathematica*'s built-in partial differentiation operator never had to be invoked. Additionally some ideas of exterior algebra had to be represented and some polynomial invariants using determinants. Otherwise, polynomial simplification, factorization, and root finding (all built into *Mathematica*) were the only other constructs needed. One big advantage discovered by using symbolic computation in this way was that *formulae* for the solution of geometric problems could be developed and then used in numerical computation (for example, in creating graphical illustrations).

This approach to classical, plane projective geometry is quite satisfactory, as the symbolic polynomial algebra really gives a middle way between synthetic and analytic geometry. Several difficult problems remain to be investigated, nevertheless:

- (1) How to extend this approach to fields of *characteristic p*?
- (2) How to make good use of symbolic computation with *algebraic numbers*?
- (3) How to refine the methods to apply to the *real projective plane*?
- (4) How to move to *higher dimensions* beyond plane geometry?
- (5) How to use of modern methods of symbolic computation in *ideal theory*?
- (6) How to connect with *automated theorem proving*?

As regards that last point, *Mathematica* — by doing algebraic transformations and symbolic rewriting — *does* actually prove theorems. However, these proofs are not fully automatic, as the user of the system has to issue the commands to be carried out. The user also has to supply the interpretation of the results, often resulting in a non-automated search for significant output. Nor does the system formulate conclusions by itself. Still, the experience was encouraging in that the algebra generated could not be carried out with any pleasure *by hand*, and the use of the computer definitely helped the author to *think*. The developments by Edmund M. Clarke (CMU) and his associates on the implementation in *Mathematica* of the *Analytica* program indicate that a certain kind of marriage between computer algebra and theorem proving can be very effective. More developments of this kind are needed, however, before courseware can be written that uses theorem proving.

□ Schedule and Syllabus for Introduction to Modern Mathematics with *Mathematica*

The bread-and-butter, service course for first-year students, blandly called **Introduction to Modern Mathematics**, is really a review of some basic algebra (not often well taught anymore in high schools in the USA), an introduction to number theory, a brief introduction to logic, and a glimpse of some topics of discrete mathematics *via* some problems of counting — which as well provide many good examples for doing proofs by mathematical induction. The major objective, of course, is to encourage abstract thinking. Unfortunately, the course curriculum is not well integrated with other courses in Mathematics and Computer Science, in the author's opinion, and so it does not fulfill these purposes as well as it should in the present curriculum.

The reason for attempting a *Mathematica*-based version of the course was to take advantage of elementary programming in problem solving, to allow for the exploration of many examples, and to illustrate results with computer graphics. The author feels he can argue that these objectives were achieved. There is also a hope that students, after becoming familiar with *Mathematica*, will use the software in future courses.

During the two semesters of the academic year '95/'96, the course was organized as two large lectures per week for all the students and then two labs in groups of about 20 students, each with their own computer. It was very helpful that the University and the School of Computer Science remodelled four computer classrooms in the summer of 1995 which could be used for this course (but they are primarily for the teaching of introductory programming, though they are also used for other courses). Further teaching facilities are being prepared now during the summer of 1996.

The outline of the course as delivered by the author as Lectruer in the last spring term is as shown below. The laboratories we handled by a group of five Section Leaders. There was also a voluntary Help Session run by Dr. Miller every Friday afternoon to assist with homework. The computer clusters were open evenings and weekends for students to work individually. (A few students had their own computers.) The examinations were administered in the clusters.

Class organization:**NUMBER OF STUDENTS** (Spring '96) Approx. 100.**LECTURES** (50 min.) Twice a week.**COMPUTER LABORATORIES** (1 hr. 20 min.) Twice a week.**HOMEWORK ASSIGNMENTS** (14) and **QUIZES** (12) Once a week.**MIDTERMS** (1 hr. 20 min.) Three Midterms.**FINAL EXAMINATION** (3 hr.) At end of course.**Grading:**

HW	MTs	Qu	Final	Total
25%	30%	20%	25%	100%

Lecture schedule:

WEEK 1.	2 Lect.	[Mathematica, Lists, and Primes]
WEEK 2.	2 Lect.	[Quotients, Remainders, and Digits]
WEEK 3.	2 Lect.	[Recursion, Factorials, Permutations]
WEEK 4.	2 Lect.	[Induction and Well Ordering]
WEEK 5.	2 Lect.	[Structural Induction]
WEEK 6.	2 Lect.	[Iteration, Periodicity, and Rationals]
WEEK 7.	2 Lect.	[Irrationals and Cycles]
WEEK 8.	2 Lect.	[Iteration and Chaos]
WEEK 9.	2 Lect.	[Boolean Functions and Automata]
WEEK 10.	1 Lect.	[Closure, Periodicity, Iteration]
WEEK 11.	2 Lect.	[Relations and Functions]
WEEK 12.	2 Lect.	[Relational Operations and Permutations]
WEEK 13.	1 Lect.	[Structure of Permutations]
WEEK 14.	2 Lect.	[Equivalence and Congruence Relations]
WEEK 15.	2 Lect.	[GCD, LCM, and the Euclidean Algorithm]
TOTAL	28 Lectures	

Course Syllabus:

WEEK 1. **[Mathematica, Lists, and Primes]** Demonstration of the *Mathematica* Frontend and the use of Notebooks; examples of numeric and algebraic expressions, and other details of syntax; introduction to lists and arrays and some basic programming constructs through examples; introduction to prime numbers.

WEEK 2. **[Quotients, Remainders, and Digits]** More on lists; integer and fractional parts; powers and positional notation; the division algorithm; the radix representation of integers; conversion between systems; introduction to recursive definitions; introduction to divisibility and modular arithmetic.

WEEK 3. **[Recursion, Factorials, and Permutations]** Review of recursive definitions by example; generalizing the radix representation; the factorial representation; enumerating permutations; implementing digital arithmetic.

WEEK 4. **[Induction and Well Ordering]** Mathematical Induction; least-element and well ordering principles; other forms of induction; review of theorems requiring inductive proofs; the connection between induction and recursion; finite sets and cardinality; disjoint sums and cartesian products; counting principles. [To First Midterm]

WEEK 5. **[Structural Induction]** Integer lists and the word algebra; principles of induction and recursion; trees as iterated lists; inductive and recursive principles associated with them; Konig's Lemma.

WEEK 6. [Iteration, Periodicity, and Rationals] Doing iterations; the Accumulation Laws; periodic and eventually periodic functions in general; the radix representation of fractions; the theorem on periodic representations.

WEEK 7. [Irrationals and Cycles] Discussion of rationals vs. irrationals and of proofs of irrationality; a brief look at continued fractions; finding cycles; examples of periodic functions.

WEEK 8. [Iteration and Chaos] Finding fixed points and roots by iteration; uses of fixed-point operators; chaos via iteration; the example of the Logistic Function. [To Second Midterm]

WEEK 9. [Boolean Functions and Automata] Representation of Boolean functions by polynomials; the enumeration of Boolean functions; sets, subsets, and indicator functions; Boolean set operations; vectorized Boolean functions; the application to cellular automata.

WEEK 10. [Closure and Periodicity under Iteration] Closure of a set under a function of list of functions; finite closures and cycles; examples of number-theoretic functions producing cycles; the notorious $3x + 1$ problem; cycles for automata iterations; demonstration of multi-state automata.

WEEK 11. [Relations and Functions] The idea of a (binary) relation; properties of relations; functions as special cases of relations; one-one vs. many-one functions; relation tables and Boolean matrices; picturing relations.

WEEK 12. [Relational Operations and Permutations] Relational product; representation of relational operations by matrices; transitive relations and the transitive closure; representing and computing permutations; isomorphism of relations. [To Third Midterm]

WEEK 13. [Structure of Permutations] Compositions and inverses of permutations; defining the order of a permutation; finding the cycle structure; computing the order of a permutation; isomorphic permutations.

WEEK 14. [Equivalence and Congruence Relations] Equivalence relations; equivalence classes; equivalential closure; congruence relations in the integers; combinations of congruences.

WEEK 15. [The GCD, LCM, and the Euclidean Algorithm] Defining the GCD and the LCM; relatively prime pairs of integers; properties of divisibility; the Extended Euclidean Algorithm; the prime factorization of integers; the infinitude of the primes.

□ Commentary

The ordering of the topic was not really satisfactory in the Spring '96 version of the course. At the start of the class the students must not only hear about new ideas, they must also become familiar with some basic features of the *Mathematica* language. It is not easy quickly to achieve both aims. The author attempted an introduction via a large suite of examples. Many of the students were confused and kept asking what was the “theory” behind the examples. The meant, apparently, that they did not have enough instructions about what to do with the examples and sufficiently many reasons why they should remember them. However, *number representations* (standard

radix, factorial, Fibonacci, and binomial) and discussions of rationals and irrationals and elementary properties of continued fractions provide a very fertile field to connect number-theoretic and algebraic properties with questions of algorithmic computation and the use of data structures. The author feels very firmly that the topics related to representations provide a suitable and sound entry point to discrete mathematics — provided that the examples and exercises are interesting and comprehensible.

Because the students became very restless with the introduction last spring, the author moved perhaps too quickly to the discussion of the various forms of *proofs by mathematical induction* and *definitions by recursion*. It would have been better to combine some of this with the topics of *congruence relations* and the *Euclidean algorithm*, which in one way or the other were postponed to the end of the course. Congruences and properties of divisors give excellent occasions for proofs, and the properties are important for the understanding of many mathematical examples concerning finite structures as well as computational algorithms. Thus, these sections should definitely have entered the lecture program earlier.

The author finds that the topics of *iteration* and the finding of *cycles* in iterative processes give a wide scope not only for proofs and algorithms but also a good way to introduce ideas about functions (injective, surjective, and so on). Also, when applied to complex data structures, there are many interesting questions about counting that come up. Iteration is a basic concept both for mathematics and computer science, and it should have a key role in such a course.

The problems of transforming vectors leads very naturally to discussions of *Boolean functions* and *set operations* via Boolean vectors used as *indicator functions* of subsets. *Mathematica* allows a very easy vectorization of Boolean functions if one uses standard polynomials. One may define a Boolean polynomial as a (multivariate) polynomial that takes on the values 0 or 1 when 0s and 1s are substituted for the variables. Then, there is an easy discussion of which algebraic operations take Boolean polynomials to Boolean polynomials. (For example, multiplication $f \cdot g$ is a good operation, but addition $f + g$ has to be replaced by the compound $f + g - f \cdot g$.) Indeed, all of standard *Boolean algebra* (including valid equations and normal forms) can be treated as a chapter of polynomial algebra; and *Mathematica* can make all the transformations of polynomials needed using only very simple definitions (including a replacement rule for getting rid of unwanted exponents in Boolean polynomials). This use of standard polynomials and polynomial manipulations has three advantages: (a) the operations automatically *vectorize*, (b) the idea *generalizes* to n -valued functions in a natural way, (c) it is not necessary to introduce *abstract algebraic structures* on other sets besides the integers (and rationals and reals) to do Boolean algebra. The author strongly feels that a first-year (even first-semester) course is *not* a good place to introduce abstract algebra. Instead, work with Boolean values and modular arithmetic can be the right *preparation* for the algebra to be studied in a *subsequent* semester. (Provided, of course, that the departmental organization of the curriculum allows for a meaningful linking between course syllabi — something that is not easy to arrange.)

The vectorization of Boolean functions then immediately applies to the implementation of examples of *cellular automata*, which bring up many questions of counting and iteration. *Mathematica* also makes the *graphics* needed

for illustrating the histories of cellular automata quite efficient and accessible. Owing to time restrictions, only *one-dimensional* cellular automata were treated in this course; in another course, two-dimensional automata were implemented just as easily. Cellular automata provide many good problems to encourage experimental computing and the making of conjectures.

From vectors and automata it is a natural step to *matrices* and to *permutations*. This is not at all the course to do linear algebra (though it would be reasonable if students took that course at nearly the same time). However, it is not difficult to introduce *Boolean matrices* as *tables for representing relations*. The Boolean polynomials apply to matrices (lists of lists) just as well as they apply to vectors (lists). Thus, the logical operations on (finite) relations become immediately computational (and the understanding of why this is so is a good mathematical exercise).

However, the polynomial $A + B - A \cdot B$ (where $A \cdot B$ is the termwise product of matrices) is not the best way to compute the union of relations when the matrices are large (especially for unions of several terms). What the author realized in making up exercises is that computationally (if not algebraically), the compound combination $\text{Sign}[A_1 + A_2 + A_3 + \dots + A_n]$, where Sign is the standard *signum function* reducing numbers to ± 1 or 0 and working termwise on matrices, is more efficient when many iterations are needed. Moreover, $\text{Sign}[A \cdot B]$, where $A \cdot B$ is now the usual matrix multiplication, is exactly the matrix representing the *relational product* of the relations represented by the two given matrices. This idea of relational product and the use of matrix multiplication (including the counting interpretation of $A \cdot B$ itself) can be explained in elementary terms (and leads naturally to ideas of graph theory — in a later course). And it is quite computationally efficient. Thus, even for quite large matrices, it is reasonable to solve — by fixed-point iteration — the matrix equation $X == \text{Sign}[A + X \cdot X]$, where A is a given square Boolean matrix, in order to find the matrix representing the *transitive closure* of the relation represented by A .

Relations require many pictures for their understanding: both line (or arrow) digrams and various kinds of tables. Of course, *ordering relations* and *equivalence relations* and the basic properties of transitivity and symmetry should be covered through many examples. What of *permutations*? Should these be left to the algebra course? *No, definitely not!* Many students will not take the algebra course, but more positively the idea of a permutation is too basic to be pushed into the future. This introduction is not a group-theory course, but *compositions* and *inverses* of permutations are something to be understood without the axiomatics of groups. In particular, all the properties of relations encountered in the elementary examples immediately bring up the question of *isomorphism* of relations — something to be explained using permutations. Iteration of functions leads to finding permutations. But permutations are functions, and iteration is essential in understanding the cyclic structure of permutations. And, working with permutations brings up many problems to be solved by counting and induction.

In Spring '96 the author did not have time to tie up fully enough the connections between permutations and equivalence relation (*via partitions of sets*) and the very important proof that every permutation is either *even* or *odd*. He strongly

believes this course should contain that theorem. And he believes that the integrated use of computing in dealing with (large) permutations makes the concepts and theorems far more meaningful to students than can be done in the limited time available by paper-and-pencil definitions and calculations.

The work in preparing the course and giving it in a computer-based form was very, very painful and time consuming, but the attractiveness and appropriateness of the the topics has convinced us — Scott, Miller, and Sutner — that whatever the future of computer-based teaching at CMU this course should be published quickly in some form involving both print and electronic files. We plan to work together on this project over the coming academic year.

■ 3. The Lessons Learned

The lessons learned by the author over these several years of experience can be conveniently grouped as answers to Professor Steen's queries (for which he no doubt has many answers himself).

□ *Can computers help students understand mathematics? Can students develop mathematical intuition without performing extensive mathematical manipulations? Do the mechanics of computing obscure mathematical insight? Will using computers reduce students' facility to compute by hand?* This is really all one big question. Let us remember that by themselves, computers can do nothing. It is possible we will see in the future self-study tutoring programs which will automatically guide the student through a subject. But the success of tutoring programs for college-level courses will take a terrific amount of development — both in computer software and in course planning. We are not ready to do automatic teaching yet, even though there are some good examples available for some elementary courses.

The present focus of work should be in getting good materials into computer-held form. It is easy to give a single demonstration lecture, but it is quite hard to get 15 weeks of course materials into the computer. Computers can only help students understand mathematics if the mathematics is *in* the computer. And it also takes a lot of thought to construct good examples illustrating the mathematical ideas.

As regards manipulation, what has to be done with the computer is not to use it as a magic, black box (and one using black magic at that), but to break the calculations into phases so the student can see what manipulations are necessary. This should not be a passive activity either, and the student should be given suitable control over choosing manipulations to be done by the computer. The way intuition is gained is through a combination of *success* and *failure* in many examples, where the student is prodded to ask why things are working out — or not working. If this analytical thinking is not encouraged, then the computer will be just a magic instrument provided by the witch doctor. What the author believes should be enhanced is the thinking processes, and whether *extensive* hand calculation is needed for this is a question that only very subtle psychological testing could answer. Success in problem solving is probably a more valuable experience than being able to do hand calculations.

A side comment here may be relevant. Nearly 20 years of using word-processing programs almost daily has improved the author's spelling considerably. Something happened to him in elementary school to make spelling a pain and a chore. However, being able to correct things quickly (not automatically, but through asking the checker to check and the computer-held dictionary to define) has done what the school teachers could not. The key feature is the quick correction. The student has to be able to check answers quickly, and if something is wrong make a change while the problem is uppermost in his mind. This quick response is one of the main advantages of computers — provided the information is there to access. By seeing the correct answers in many contexts, learning takes place. Again, how to assess the extent or efficiency of the learning is again a (difficult) problem for psychologists. The experience the author had in the several versions of the two courses described in the last section, however, convinced him that a program such as *Mathematica* together with preparation of examples can answer questions very quickly and on a scale impossible with hand calculation. Therefore, he is convinced of the value of using computers.

There is a serious danger, nevertheless. The teacher may know very well why things work, and he may call up the computer to give the final answer without showing any *intermediate steps*. It is all too easy to hide things by computer and leave connections invisible. The teacher always has to ask himself whether he is showing enough to make the results understandable and plausible. Computers can be misused in numberless ways, and one must constantly ask whether the answers obtained are reasonable. If there is any doubt, a stepwise analysis must be carried out to be sure that the computer is doing what was intended.

□ *How does computing change what students should know about mathematics? How does computing change what students can learn about mathematics? Where in the curriculum is computing most appropriate? Will use of computers reduce the need for remediation?* Will computers change mathematics? Well, yes and no. Problems are problems, and they must be solved by whatever means. But computing does very much bring home which solutions are *feasible*. For example, a massive search may theoretically solve a problem, but it may be completely infeasible even for moderately sized problems. So the questions of how solutions are carried out and why they are fast or slow is a new aspect that the computers may bring forward and which may have been missing from conventional curricula. The author feels that computers can help make receptive students learn *more mathematics*. And inquisitive student will always want to know why things are true, will always bring up something outside the syllabus, and the computing environment can help answer those questions by letting the student experiment on his own. Moreover, seeing success in one kind of problem suggests at once other problems or further generalizations. The computer often lets one adapt solutions to new cases fairly easily, or at least lets one try to see why a previously usable method fails to be applicable.

Another important feature of the computer is its memory — its *retentiveness*. When paper and pencil homework is returned, does the student ever look at anything other than the mark at the top? Are the papers saved? With computers there is a potential at least for the reuse of work from assignment to assignment — if the teacher is clever enough to ask the right questions in the right way. Certainly the author has found that working out formulae exactly makes it possible to use and reuse

the results in new ways. For example, in geometry, the formulae may prove a theorem, but then they may be used in numerical computations to draw pictures. In combinatorics, the result of solving a counting problem results in a formula or program that can be put to work in many other problems. Or, the new problem may require some revisions of something done before, and if the old program is retained, it can then be adapted to meet the new requirements without having to start from scratch. Of course, as with any growing archive — whether electronic or print — some good organization is required for quick recall. The author feels that the software systems designers could make life easier for us on this score by providing better aids to the organization of computer files.

And where in the curriculum should computing enter? Well, should it be excluded from any part of the course of study or area of research? Who does not need to draw a picture or do some algebraic or symbolic simplification? The computer should become the sketchpad, laboratory, archive, and communication device for anyone who is willing to learn how to use it. True, we need some better development of *typesetting* to make the use of the computer less painful for sketching, but the author believes we will see this before the turn of the century. Already in the last two or three years, the computer network facilities have made *communication* much more effective. The author also believes that by the turn of the century we will have seen a big change in mathematical publishing, and this change will have a major impact on teaching as well.

What about remediation? Yes, having materials available in computer-held form will allow the student to review past work and past courses at his own pace. If he is motivated enough, he will find what he needs. Students may also be able to review tests and problem sets more easily in order to understand their mistakes better. E-mail can be effectively used to ask questions and to get answers. The key feature of using the computer will be to get the student to record his question — in whatever form — at the moment at which he is concentrating on the subject. With computers, this can be any time of the day or night; and with portable computers, this can be almost anywhere (except, perhaps in the swimming pool or dance floor). But whether the need for remediation will be *reduced* is not a question we can answer today. We should concentrate in the short term in making remediation easier — by capturing courses in computer-held form. How many times have professors not even been able to find the syllabus for a course from a previous year, let alone copies of the tests and problem sets?

□ *Can colleges afford computers for all mathematics students? How much time and distraction is computing worth? When will there be good software and compatible hardware? Can textbooks ever reflect contemporary computer examples?* The economic factor is very important. Budgets are tight, and higher education is suffering from a reduction of state funds in every country at the present time. But the author asks: Can colleges and universities afford *not* to supply computers to students (perhaps for purchase at favorable prices)? Can colleges afford *not* to provide the service necessary to make educational computing effective? It is not enough just to buy a bunch of computers and tables and chairs and to plug the machines in. It is not enough just to buy various software packages and to install them. Students and staff have to be trained to use the software, and both the machines and the programs require constant maintenance. Some of the training

and care can be done on an institution-wide basis. But there are problems that come up every day that need immediate attention, and departments have to have some dedicated staff to help solve them. Professors cannot be expected to become computer experts capable of doing intensive trouble shooting. Professors who are going to make use of computers — and there will be more and more of them — have to be sufficiently computer literate to make the most of the equipment, and they have to have some technical insight to be able to recommend and oversee the changes that will be necessary nearly every year. But professors change, their assignments change, and their interests change. Therefore, there has to be a kernel staff to keep things running smoothly from year to year. This staff has to understand something of the course requirements and the teaching problems. Different departments probably will have different demands and needs, so some staff has to be attached to each department. Some staff members can be instructors in some courses or course organizers for other courses, but there have also to be technical people who can really care for the machines and software. Not every institution has yet realized this need for staff commitments.

And why can computers be not avoided? Because computers now enter into all parts of business and government — not always very economically to be sure, but they are there to stay. They will come into more and more homes very soon. The truly amazing growth of the Internet (and the crazy “investment bubble” in internet companies!) has made everyone aware of the (some) of the potential of computers for conveying information. Many colleges are already making good use of educational computing. If others do not catch up, they will loose out to competitors. In 1996, private university education now costs something on the order of **\$1000** per week in the United States. At this price, students will soon realize that they are being cheated if they are not being provided with the opportunity of using computers in their coursework. The question of “time and distraction” can no longer be avoided — colleges have no choice. The problem is how to keep costs in hand. But economy cannot be achieved by minimizing expenditure on staff!

Software, hardware, and textbooks? Everyone is happy to complain about them. But it is in fact the case that more and more textbooks are including computer supplements — either through disks in the books or through Internet archives maintained by the publishers. The author feels that at last the problem of paying for Internet products is being solved, and that publishers will find electronic distribution of certain materials both helpful and appropriate. As far as hardware goes, it is a sad fact that the higher-education market is not large enough to make much impact (witness the awful failure of the NeXT computer and the more recent sad decline of the Apple Macintosh). The author has personally no desire to convert to Microsoft Windows, and only hopes that such operating systems as Linux, which can run on many different machines in a uniform way, will make it possible to develop a sound and reliable educational computing environment. Software suppliers must make their products available for this operating system, however. Whether they will or not remains to be seen.

□ How much programming should be taught in mathematics courses? Can pure mathematicians convey an appropriate computational perspective? How will new faculty fit into computer-enhanced programs? Will use of computers improve teaching of mathematics? The author has heard many mathematics professors say, “I will not teach program-

ming!” Fortunately, not all engineering, physics, and computer science professors are refusing to teach mathematics, since mathematics departments are not always teaching the necessary parts of mathematics in the right way for the more applied areas. It is not just the Calculus: something as basic as *Linear Algebra* can be a very sore point at some institutions.

What the author believes is that if the professors are willing to use programming, then sufficient programming can be learned by students *by example*. The point is that the examples have to be there to be used as models. The people who know how to do it should thus produce more materials for wider distribution. The newer faculty will catch on quickly, but the older faculty may be the ones showing the most resistance. Much of the design and understanding of programming examples is mathematical in nature, and “pure” mathematicians are just as capable of conveying a computing perspective as are the computer science professors. Indeed, the mathematicians may have an advantage in knowing more examples, more proofs, more tricks, and generally more facts than some of the staff that have had to concentrate on technological problems. It takes all kinds, and we should not make more efforts to erect more barriers between subjects and waste time on “protecting” turf. If we do, we are cheating our students in regard to obtaining a wider perspective on the connections among the sciences.

Not all subjects can immediately benefit from the use of computers. But much of algebra, geometry, number theory, numerical analysis, and combinatorics certainly can. And computation, as has been asserted before, often brings up new and mathematically interesting questions (consider cryptography, for example). Would anyone want to teach about differential equations and dynamical systems today without showing what computer simulations and computer graphics can convey about the qualitative descriptions of systems? It is ardently to be hoped not. Perhaps *chaos* has become too much of a popular buzz-word, but it would not have caught the popular imagination without computer graphics. Mathematicians should take more advantage of this publicity, since it is notoriously difficult to give popular descriptions of abstract mathematics. Inasmuch as student numbers are decreasing, why not try to attract more people to the study of mathematics by making it more exciting? For example, the author believes that a course in complex variable theory could be made quite interesting by using computers (say, with *Mathematica*), but unfortunately he has never had the opportunity to try this out. The point is that a textbook can contain only so many pictures. With a computer, hundreds of pictures of difficult transcendental functions can be made. Complex functions seem to be an ideal subject for mathematical lab work.

□ *Won't computer packages for calculus lead, as they have in statistics, to much meaningless calculation? If computers handle routine calculations, what will students do instead? What are appropriate prerequisites for computer-based calculus courses? Should mathematics be a lab science?* The author does not know how much “meaningless calculation” has gone on in Statistics, but that subject certainly *needs* computers! It is the responsibility of the instructors to provide meaningful problems that take full advantage of computers. It is no longer necessary to “cook the parameters” so the answer turns out to be $1/3$ these days. It is no longer necessary to keep all algebraic expressions down to degree 4 or lower. It is no longer necessary to slog through

mammoth tables of integrals. But it is necessary to know what you are doing just to avoid meaningless activities. The elimination of the routine can free the students to explore and to think, but they need to be shown how to make use of the computer as a tool in this way. It takes work to do this, and departments need to find ways to make it possible for interested professors to do this necessary work.

Does a computer-based calculus course need special prerequisites? The author would guess that Bill Davis (Ohio State University), Horacio Porta, and Jerry Uhl (University of Illinois at Urbana-Champaign) who designed over many years a course “Calculus & *Mathematica*” (published in 1994 by Addison-Wesley Publishing Company, Inc.) would say not. It does not take all that long (perhaps about three weeks) to become comfortable with the computer, and then the computer is *transparent* — all the emphasis can be put on the subject matter. Davis, Porta, and Uhl said in one of their introductions to their course:

“Just as arithmetic is the introduction to *counting*, calculus is the introduction to *measurement* — both exact and approximate. Calculus is a tool-kit of measurement devices: derivatives, differential equations, definite integrals, series expansions and approximations, data fitting, linearizations, vectors, dot products, line integrals, gradients, curls, and divergences. Today’s popular texts present most of the tools but end up focusing too much on the tools and not enough on their *uses*. The successful calculus course produces students who recognize a calculus situation and who know what calculus tools to bring to bear on it. This is the goal of Calculus & *Mathematica*.”

The author has not taught from their (extensive) materials and can offer no well-founded judgement as to whether they have truly reached their goal. Brief examinations of the text and computer notebooks, and exposure to presentations by those authors about their course suggests that they have. But the only real proof is in actually teaching the course. In any case, they expect that students will be as ready to start their course as any standard calculus course.

And what about *labs*? Davis, Porta, and Uhl also said:

“Why has the availability of computers and calculators not yet resulted in significant changes in mathematics instruction? Computer advertisements indicate that driving a truck full of computers to school and unloading them at the mathematics classroom door will result in miracles. But, in case after case, high expectations have dissolved when the computers failed to deliver the expected miracles. The fault lies not in the computers themselves. The fault lies in the fact that the already overburdened individual instructor has to figure out what to do with the machines. The expedient and usual response has been to add a lab onto an existing course to allow the students the opportunity to do problems from the text by machine instead of by pencil and paper. This response may stymie the students’ thinking process. Students have been taught, in the lecture part of the course, to do the procedures of mathematics by hand. Then, in the lab, they are shown how to do the same mathematics automatically by computer. They may think: Why learn the hand operations when the computer does them automatically?”

What they caution about here is the *disconnecting* of the lab from the main thread of the course. Mathematics can only be a lab science if the lab can be used for exploration — and it has to be set up to do so. In planning a course one also has to ask:

How much lecturing is called for? Davis, Porta, and Uhl have their own answer:

"In the typical model of C&M instruction, new material is never presented in lecture form. Students meet with their instructor once a week for a discussion of the week's assignment and the relevant Literacy Sheet. There is an implementation of C&M at sixteen rural Illinois high schools in which the only discussions are among the students themselves, because the schools cannot fund full-time instruction in calculus. Anecdotal observations indicate that C&M instructors who insist on introducing ideas via lecture find that difficulties increase in direct proportion to the size of the lecture component of their course. Perhaps C&M students have a lot in common with Winston Churchill who said: *Personally I am always ready to learn, although I do not always like being taught.*"

In other words for, those authors it is **all labs! and no lectures!** This professor does not agree. But, giving good lectures is as difficult as giving good labs. Undoubtedly different courses will need different arrangements.

The experience the author had in the Spring of 1996 in giving lectures was, however, **very discouraging**. It took a lot of physical effort to set up the computer for each lecture. The computer projector was not bright enough to use without turning out most of the lights in the large lecture room. When the lights were out, a small desk lamp was necessary to be able even to see the computer keyboard. The class was too large. The air conditioning had been turned off in an attempt by the University to save money (under the assumption that no cool and fresh air was needed during the winter). Lots of students went to sleep during lectures. The geometry of the hall made it difficult to have contact with the students and to answer questions. It was impossible to write on a blackboard or use an overhead projector while the computer projector was on. As a result of this torture, the author will refuse to lecture in that room ever again if a computer is to be used! Institutions must give more attention to the design of classrooms.

The students complained that the lecturer from time to time was "just showing them how to use *Mathematica*." This was not true. What the lecturer was trying to show then was that it was **possible** to use *Mathematica* — in hopes that the students would do it themselves. Some did; others copied homework from the more enterprising. Since the material for the course was continuously undergoing a complete revision during the term, the lecturer made the mistake of putting **too much material** into the lectures. This was the real reason, he thinks, why the students were not getting the ideas: there was too much to take in every session. Now that the course has been captured in a computer-held form, it would be possible to segment the topics better to make the work of each week more focussed and more enjoyable. But there is still a nagging question about lectures.

It is quite possible that a solid 50-minute period for a lecture to first-year students on mathematics is not advisable. The lecture time should perhaps be broken up into various activities. Or a schedule of three 80-minute directed lab sessions (in smaller groups) including some lecturing, some question answering, some individual problem solving, and some exam post mortem would be more effective. That requires more instructor manpower, however. Or perhaps the Illinois model of no lectures whatsoever is the best plan. How can we decide? In the first place the class materials

needs to be prepared and tested by use. Then, some kind of controlled experiments have to be run, putting groups of students into different environments. But who is going to do this, and who is going to evaluate the results? What are the criteria for evaluation. Most professors and most departments do not have time (or energy) for elaborate experiments. Even worse, experiments in teaching are not projects leading to tenure for younger staff — who are often the most lively and with the best rapport with the first-year students. And so we bumble on from year to year, sometimes passing through a crisis of “reform” and often letting the pendulum swing back and forth under its own momentum. But the computer is catching up to us, and many of us will soon be out of date and out of place in a new environment of communication.

■ 4. Some Conclusions and Suggestions

The author has often been asked: *Why use Mathematica? Is it not too expensive and too unreliable? And as a programming language is it not too unstructured and inefficient?* The exaggeration of its reputation for unreliability is overdone in the author's opinion; however, when the new version 3.0 is released this year some very serious reviews must take place, since Wolfram Research has now had a substantial period to correct and refine the program. The expense of the software is indeed a problem, and again a review should be carried out to determine whether the investment is worth the results. But many books now use Mathematica programs and examples, and one has to ask whether a college can afford not to subscribe.

The author found the value of the software to be in the combination of these features: *Mathematica* is at the same time

- A programming language
- An interactive computing system
- A symbolic-numeric-computation engine
- A text-processing program
- A presentation manager

Perhaps **Maple** could fulfill all these requirements, but the author found that the ease of implementing *rule-rewriting systems* in *Mathematica* led to many applications that students could be taught to do for themselves. But again, some criteria for evaluation of features is needed.

Wolfram Research is promising a revolution in typesetting that will bring *two-dimensional formulae* to interactive computing. We shall have to see how it works. What is definitely needed for the future is better *file management*. If one has only five files (of modest size), one can remember fairly well what is in them. However, when the number of files gets up to 25, and when several of the files are changing every day, it becomes impossible to keep track of what is where. *Mathematica* 3.0 will make notebooks *Mathematica objects*, and so it will become possible to write *Mathematica* programs that process notebooks. This should make dynamic indexing feasible, but it may take some third-party developments to create environments in which courseware development is less time consuming and vexing. As it is, the text-processing capabilities of the current version of *Mathematica* (Version 2.2.2) — which on the Macintosh we already available in 1.0 — were absolutely essential to the author for writing course notes and lectures

notebooks. It takes a lot of writing to put a course in shape, and the computer has to make the writing as easy as possible. We can thank Theo Gray for doing an excellent job with the *Mathematica* Front End from the start, and we can look forward to improvements and enhancements. And, to be sure, improvements (especially in printing) are to be desired.

As for programming, there are many dilemmas. *Mathematica* is an interpreted language, and so it can become very slow on larger problems. There is a facility to link Mathematica to compiled programs in the C language, and Klaus Sutner, for one, has made very good use of this facility in implementing algorithms for processing finite automata with large numbers of states. Unfortunately, there are still some problems in using the linking facility with some operating systems, and more implementation problems have to be solved. Also, the creation of environments that can be used by less expert programmers has to be faced. This is one sort of programming problem. Another problem is that Mathematica is an untyped language. This makes some programming easier, but it makes other kinds of programs that require the dynamic checking of features of data structures harder. Some thought should be given to finding a middle way of constructing programs. Or, perhaps, another generation of implementations of symbolic computation software will allow for better compatibility with "industrial grade" programming languages. In the meantime, the author has found that *Mathematica* is a satisfactory language for *prototyping* algorithms for didactic purposes, and so he hopes that he is making students think. If we can give them better tools in the future, so much the better.

Another criticism often heard in using software is: *The medium is becoming the message*. The author both resents and rejects this criticism as regards his own course developments as explained earlier. For him the medium is definitely not the message! But we have to remember that any medium should be well chosen to get the message over well. If one does not understand the medium, however, the message may be wrong. And if one does not understand the *scope* of the medium, the message may be too simple. Therefore, it is necessary to become fairly expert in the medium in order to get the best results. And the students must gain suitable expertise as well to be able to do their work. The makers of software need to keep this problem in mind and provide suitable guides, so that new users can obtain the necessary level of expertise as quickly as possible. One guide (or handbook) may not be suitable for all possible users.

As a medium for mathematics, the computer can truly become *microscope* and a *telescope* to see mathematical phenomena invisible by ordinary means. But the equipment and methods needs good "tuning" so that the "pictures" are neither blurred, distorted, or irrelevant. When the "focus" is good, the coursework and publications already available in many different mathematical subjects do — in the author's considered opinion — show that the money and effort is worth the gain. But we need to do more with the use of computers, and we need to do it better.